

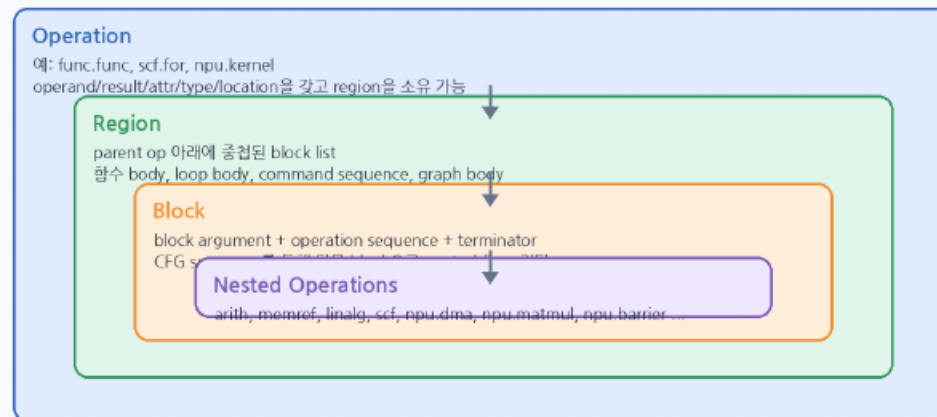
MLIR NPU Compiler

Lecture 03

Region 과 Block: NPU kernel / command region pseudo IR

- Operation 내부의 region 을 읽는 법
- Block argument와 terminator로 control flow 해석
- DMA/compute/barrier command region 설계

MLIR IR Nesting: Operation -> Region -> Block -> Operation



오늘의 위치

2 강 Operation anatomy 에서 3 강 Region/Block 으로 확장

Course flow

- 2 강 : Operation / Value / Type / Attribute 읽기
- 3 강 : Operation 안에 중첩되는 Region 과 Block 읽기
- 이후 강의 : SCF/Affine loop, Pass, Rewrite, Bufferization 로 연결

Model IR

- > Operation anatomy
- > Region / Block
- > Loop / CFG / Schedule
- > NPU dialect
 - > Command buffer

Learning Objectives

- Operation -> Region -> Block -> Operation 구조를 설명한다
- Block argument 가 merge value/loop variable 을 표현하는 방식을 이해한다
- Terminator 와 successor 를 통해 CFG 를 읽는다
- NPU command region pseudo IR 을 설계한다

왜 NPU Compiler 에서 중요한가 ?

Compiler issue

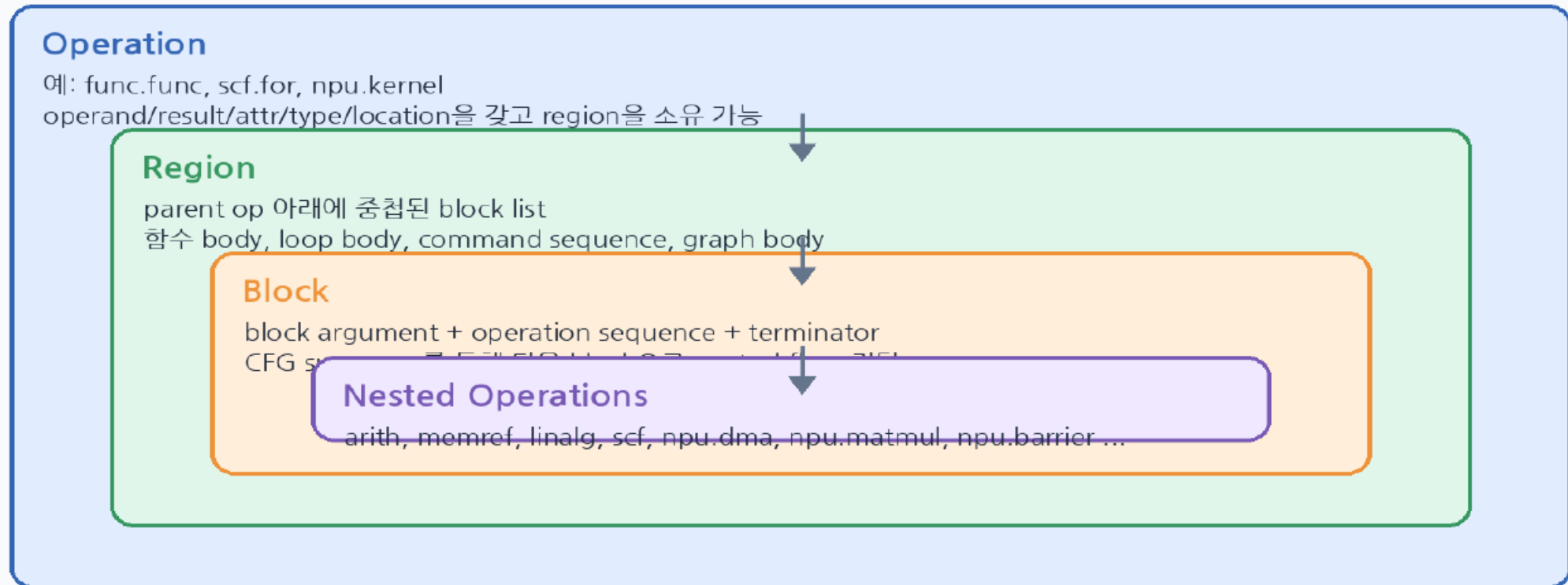
- NPU kernel 은 단일 op 가 아니라 command sequence 입니다
- DMA, barrier, matmul, store 의 ordering 을 IR 에 표현해야 합니다
- Region 은 command sequence 를 semantic unit 으로 묶어 검증 / 스케줄링 /serialize 를 가능하게 합니다

Region 으로 표현할 것

- Tiling loop body
- Async DMA dependency
- SRAM/ACC lifetime
- Runtime command buffer ABI

MLIR IR Nesting Model

MLIR IR Nesting: Operation -> Region -> Block -> Operation



Terminology Cheat Sheet

Term	의미	NPU 관점
Operation	IR 의 기본 노드	npu.matmul, npu.dma
Region	parent op 내부 block list	kernel/command body
Block	argument + op list + terminator	command step 또는 CFG node
Block argument	block entry value	tile index, token, descriptor
Terminator	control/yield/return	npu.yield, cf.br

Region 은 semantic container

핵심 문장

- Region 은 block list 일 뿐입니다
- Region 의 실제 의미는 parent operation 이 결정합니다
- 같은 region 문법이라도 func body, loop body, graph body, NPU command body 가 될 수 있습니다

```
"parent.op"() ({  
  ^entry(%arg0: type):  
    ... nested operations ...  
    "terminator"() : () -> ()  
  }) : () -> ()
```

Block Anatomy

Generic block shape

```
^block_name(%arg0: type, %arg1: type):  
  %v0 = "dialect.op"(%arg0) : (type) -> type  
  %v1 = "dialect.op"(%v0, %arg1) : (type, type) -> type  
  cf.br ^next(%v1 : type)
```

읽는 순서

- Block arguments: block entry 에서 보이는 SSA value
- Operations: 순서 있는 op list
- Terminator: successor 또는 parent op 로 제어 전달

Entry Block 과 Region Argument

- Region 의 첫 block argument 는 region argument 처럼 취급됩니다
- func.func 에서는 함수 인자와 연결됩니다
- scf.for 에서는 induction variable 이 loop body block argument 입니다
- custom npu.kernel 에서는 tensor descriptor, tile index, runtime config 가 될 수 있습니다

Terminator 와 Successor

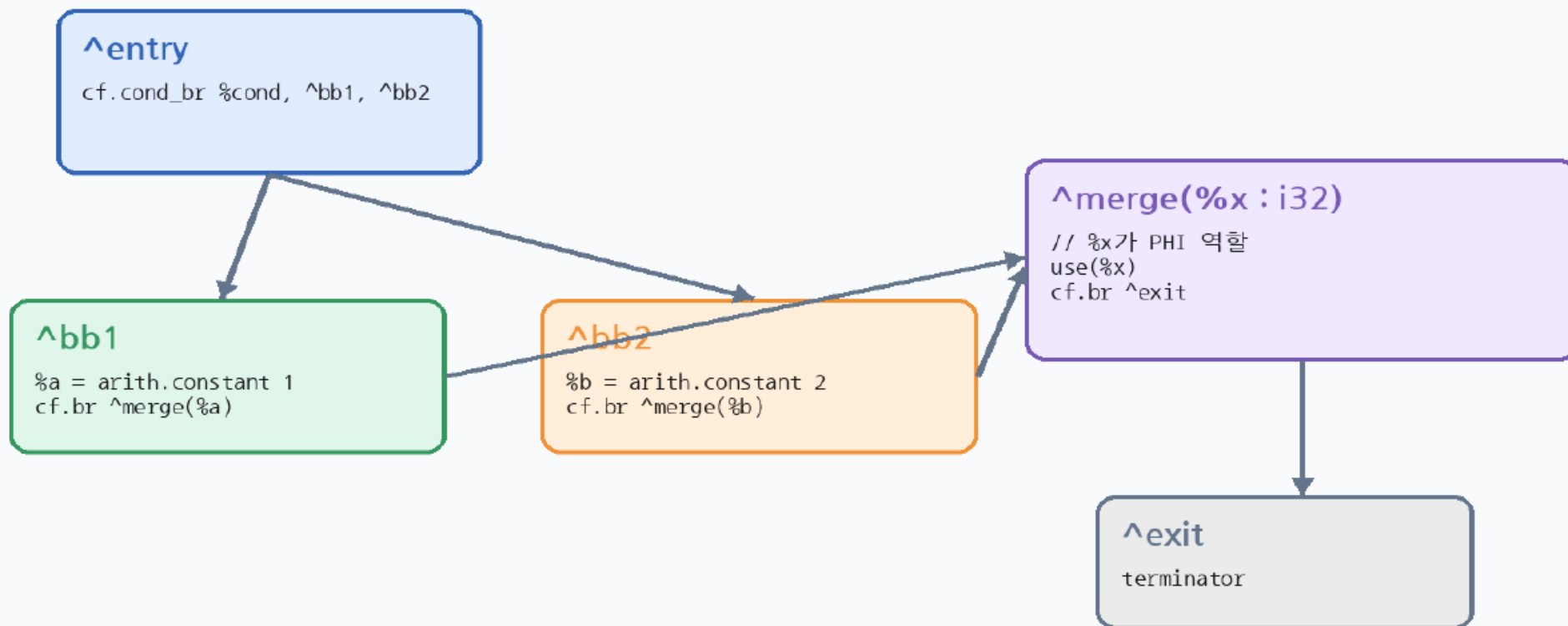
```
cf.br ^merge(%a : i32)
cf.cond_br %cond, ^then, ^else
scf.yield %next : f32
func.return %result : tensor<...>
    "npu.yield"() : () -> ()
```

Verifier 관점

- Block 끝의 terminator 가 control flow 를 결정합니다
- Branch terminator 는 successor block 과 전달 값을 지정합니다
- Yield/return terminator 는 parent operation 으로 값을 돌려줍니다

Block Arguments vs PHI

Block Arguments: MLIR의 PHI-node 대체 표현



핵심: predecessor가 successor block으로 값을 넘기고, successor block argument가 merge value가 됩니다.

Single-block Region: scf.for

```
%sum = scf.for %i = %c0 to %c1024 step %c1
  iter_args(%acc = %zero) -> (f32) {
    %x = memref.load %A[%i] : memref<1024xf32>
    %next = arith.addf %acc, %x : f32
    scf.yield %next : f32
  }
```

읽기 포인트

- %i: loop body block argument
- %acc: loop-carried block argument
- scf.yield: next iteration/result로 전달
- NPU tile loop 의 기본 형태

Multi-block Region: execute_region

```
%r = scf.execute_region -> i32 {  
  cf.cond_br %cond, ^then, ^else  
  ^then:  
    %a = arith.constant 1 : i32  
    cf.br ^merge(%a : i32)  
  ^else:  
    %b = arith.constant 2 : i32  
    cf.br ^merge(%b : i32)  
  ^merge(%x : i32):  
    scf.yield %x : i32  
}
```

주의

- Unstructured CFG 를 region 안에 담을 수 있습니다
- ^merge(%x) 는 predecessor 값의 merge point 입니다
- NPU command 에서는 structured form 을 우선 유지하는 편이 scheduling 에 유리합니다

Dominance 와 Scope

- Region argument 는 region 내부 deep operation 에서 사용 가능
- Region 내부 value 는 임의로 region 밖으로 escape 하면 안 됩니다
- Value 사용 지점은 해당 value 가 dominance 하는 scope 여야 합니다
- NPU SRAM/ACC view lifetime 검증과 직접 연결됩니다

Region Result 와 Yield

개념

- Parent op 가 region result 와 op result 의 관계를 정의합니다
- scf.for 는 loop-carried final value 를 result 로 가질 수 있습니다
- npu.command_region 은 보통 command stream/status/token 을 result 로 만들 수 있습니다

```
%out = "npu.execute"() ({  
  ^body:  
    %token = "npu.dma.async"(...) : (...) -> !  
    npu.token  
    "npu.yield"(%token) : (!npu.token) -> ()  
    }) : () -> !npu.token
```

Region Kind: SSACFG vs Graph

- SSACFG region: block, terminator, successor, dominance 가 중요
- Graph-style region: operation order 가 semantic order 가 아닐 수 있음
- AI graph IR 과 command-buffer IR 은 region kind 선택이 다릅니다

- NPU command buffer 는 보통 deterministic order 가 중요
- AI graph/fusion region 은 dependency graph 해석이 더 중요

NPU Mapping #1: Kernel Region

```
"npu.kernel"() ({  
  ^entry(%A: !npu.tensor_desc, %B: !npu.tensor_desc, %C: !npu.tensor_desc):  
    "npu.for_tiles"(...) ({  
      ^tile(%m0: index, %n0: index, %k0: index):  
        ... DMA + compute + store ...  
        "npu.yield"() : () -> ()  
      }) : (...) -> ()  
    "npu.return"() : () -> ()  
  }) : () -> ()  
})
```

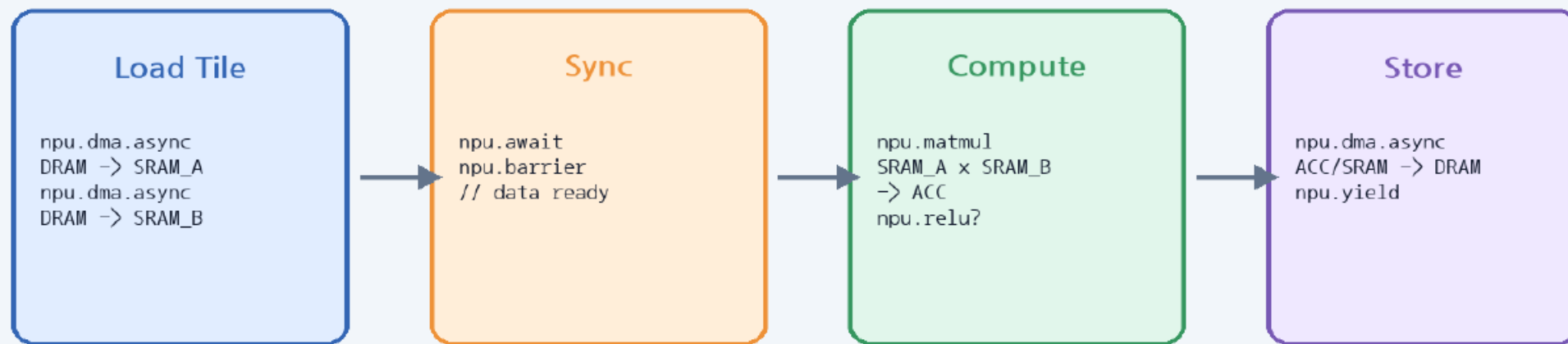
의도

- Kernel-level semantic boundary
- tile loop 와 command body 를 region 으로 보존
- verifier 가 terminator/token/memory scope 검사

NPU Mapping #2: Command Buffer Region

NPU Command Region: DMA / Compute / Barrier를 Region으로 구조화

"npu.command_buffer" region



Region을 쓰면 pass가 command sequence를 하나의 semantic unit으로 검증/스케줄링/serialize할 수 있습니다.

Pseudo IR Walkthrough

```
%a_sram, %ta = "npu.dma.async"(%A) : (...) -> (!npu.sram_view, !npu.token)
%b_sram, %tb = "npu.dma.async"(%B) : (...) -> (!npu.sram_view, !npu.token)
"npu.await"(%ta, %tb) : (!npu.token, !npu.token) -> ()
%acc = "npu.matmul"(%a_sram, %b_sram) : (...) -> !npu.acc_view
%out = "npu.requant_relu"(%acc) : (!npu.acc_view) -> !npu.sram_view
%ts = "npu.dma.async"(%out, %C) : (...) -> !npu.token
      "npu.await"(%ts) : (!npu.token) -> ()
```

NPU Region Verification Checklist

- 모든 block 에 terminator 가 있는가 ?
- block argument type/count 가 branch operand 와 맞는가 ?
- DMA token 이 compute 전에 await 되는가 ?
- SRAM/ACC view 가 region 밖으로 불법 escape 하지 않는가 ?
- compile-time property 는 attribute 로 표현했는가 ?
- command order 가 deterministic 한가 ?

Pass Design Implications

- Traversal pass: op -> region -> block -> op 재귀 방문
- Analysis pass: block argument, successor, token def-use 추적
- Scheduling pass: command region 내부 dependency graph 생성
- Lowering pass: command region 을 runtime ABI descriptor 로 serialize

Exercise A: Annotation

```
scf.for %i = %c0 to %c1024 step %c1 {  
  %x = memref.load %A[%i] : memref<1024xf32>  
  memref.store %x, %B[%i] : memref<1024xf32>  
}
```

질문

- Region 은 어디인가 ?
- Block argument 는 무엇인가 ?
- Terminator 는 명시되어 있는가 ?
- %i 의 scope 는 어디까지인가 ?

Exercise B: Merge Value

```
^then:  
  %a = arith.constant 1 : i32  
  cf.br ^merge(%a : i32)  
^else:  
  %b = arith.constant 2 : i32  
  cf.br ^merge(%b : i32)  
^merge(%x : i32):  
    scf.yield %x : i32
```

분석

- %x 는 어느 값에서 올 수 있나요 ?
- LLVM PHI node 와 어떤 점이 다른가요 ?
- branch operand type 이 다르면 어떤 verifier 오류가 생기나요 ?

Exercise C: NPU Command Region 설계

- MatMul tile 하나를 pseudo IR로 작성
- DMA A/B → await → matmul → requant/relu → DMA C → yield
- token, memory space, terminator를 반드시 포함
- 작성 후 verifier checklist로 검토

Common Mistakes

- Region 자체가 semantics 를 정한다고 오해
- block terminator 누락
- block argument 와 predecessor operand type/count mismatch
- async token 을 만들고 await 하지 않음
- SRAM view lifetime 을 region 밖으로 노출
- tile size 같은 compile-time config 를 불필요하게 operand 화

Takeaways

- Region 은 parent op 아래 block list 이며 parent op 가 의미를 정합니다
- Block argument 는 PHI-node 없는 SSA merge 표현입니다
- Terminator/successor 를 읽으면 control flow 와 value flow 가 보입니다
- NPU command region 은 DMA/compute/barrier/order 를 compiler 가 검증 가능한 구조로 만듭니다

References

- MLIR Language Reference
- Understanding the IR Structure
- MLIR Rationale: Block Arguments vs PHI nodes
- SCF Dialect documentation